

Esercizi su type inference

Esercizio 1. Determinare il tipo delle seguenti espressioni OCaml:

- A) `(3,10.2)::(3,2.1)::[]` ;;
- B) `[]::[]` ;;
- C) `fun x -> fun y -> (x,if y>0 then x else y<10)` ;;
- D) `(fun x -> fun y -> x+y) 10` ;;
- E) `let f x y z =
 if x then []
 else match y with
 | (0,0) -> []
 | (n1,n2) -> z::[n1;n2]` ;;
- F) `let g x y z =
 let k=10 in
 if x=k then y
 else [z]` ;;

Esercizi su liste e ricorsione

Esercizio 2.1. Scrivere una funzione ricorsiva `somma_positivi` che, data una lista `lis` di interi, restituisce la somma degli elementi positivi. Ad esempio, `somma_positivi [3;0;-1;2;-4]` restituisce 5.

SOLUZIONE:

```
let rec somma_positivi lis =  
  match lis with  
  | [] -> 0  
  | x::lis' -> if x>0 then x+(somma_positivi lis') else somma_positivi lis';;
```

Esercizio 2.2. Scrivere una funzione ricorsiva `somma_liste` che, date due liste `[x1;...;xn]` e `[y1;...;yn]` di uguale lunghezza, restituisca la lista `[x1+y1; ... ;xn+yn]`. Se le liste hanno lunghezze diverse, la funzione restituisce la lista vuota.

SOLUZIONE:

```
let rec somma_liste lis1 lis2 =
  if (List.length lis1 <> List.length lis2) then []
  else
    match lis1,lis2 with
    | [],[] -> []
    | x::lis1',y::lis2' -> (x+y)::(somma_liste lis1' lis2');;

oppure

let somma_liste lis1 lis2 =
  if (List.length lis1 <> List.length lis2) then []
  else
    let rec aux l1 l2 =
      match l1,l2 with
      | [],[] -> []
      | x::lis1',y::lis2' -> (x+y)::(aux lis1' lis2')
    in
      aux lis1 lis2;;
```

Esercizio 2.3. Scrivere una funzione ricorsiva `parentesi_bilanciate` che, data una lista `lis` di caratteri, restituisce `true` se la lista contiene una sequenza di caratteri '(' e ')' bilanciata. Ad esempio, `parentesi_bilanciate ['(';'(';'')';;'(';'(';'')';;')';;']` restituisce `true`, mentre `parentesi_bilanciate ['(';'(';'')';;')';;']` restituisce `false`.

SOLUZIONE:

```
let parentesi_bilanciate lis =
  let rec aux n lis =
    match lis with
    | [] -> n=0
    | '('::lis' -> aux (n+1) lis'
    | ')'::lis' -> if n>0 then aux (n-1) lis' else false
    | _ -> false
  in
    aux 0 lis;;
```